

Relatório do Hashing

Iago Cardoso Bariuka

16/10/2024

1 Descrição Geral

O sistema implementado refere-se a uma tabela hash do tipo Cuckoo Hashing, uma técnica moderna para lidar com colisões em tabelas hash. O Cuckoo Hashing implementado utiliza duas tabelas hash independentes e duas funções de dispersão distintas. Cada elemento é armazenado em um de dois possíveis locais.

2 Operações Implementadas

2.1 Inserção

A função `inserir` realiza a inclusão de uma chave (k) na tabela hash utilizando o método de Cuckoo Hashing. O índice para inserção é calculado pela função `h1(k)`, se o índice correspondente na tabela 1 estiver vazio, a chave é diretamente armazenada nesse local. Caso o índice da tabela 1 já esteja ocupado por outro valor, ocorre o seguinte processo:

- O valor atualmente armazenado na posição calculada por `h1(k)` é removido. O índice na tabela 2 para esse valor removido é calculado utilizando a função `h2()`. O valor desalojado é então inserido na posição correspondente da tabela 2.
- Finalmente, a chave (k) é inserida na posição originalmente calculada na tabela 1 pelo índice `h1(k)`.

```
1 void inserir(int k, struct Hash *table);
```

2.2 Exclusão

A função `excluir` é responsável por remover uma chave (k) da tabela hash. O processo de exclusão funciona da seguinte maneira:

- Primeiramente, a função utiliza `buscar()` se o valor retornado for `-1`, significa que a chave não existe em nenhuma das tabelas, e a função simplesmente retorna sem realizar alterações.
- Se a chave k foi encontrada na tabela 2, o valor correspondente no índice é substituído por `-1`, indicando que o local está vazio.

- Se a chave `k` foi encontrada na tabela 1, o valor no índice correspondente é substituído por `-2`, que é uma marcação especial para indicar que uma chave foi excluída, mas que aquele espaço já foi utilizado anteriormente.

```
1 void excluir(int k, struct Hash *table);
```

2.3 Impressão

A função `print-tab` tem como objetivo imprimir os valores armazenados nas duas tabelas do sistema de *Cuckoo Hashing*, de forma organizada e ordenada. A lógica dessa função é a seguinte:

- Os valores das tabelas são verificados para identificar os elementos válidos. Após a coleta de todos os valores válidos, o vetor `valores` é ordenado em ordem crescente utilizando a função `qsort`, que requer uma função de comparação (`compara`) para definir a ordem.
- Localiza o índice correspondente utilizando `buscar`. Determina em qual tabela o valor está localizado (`tabela1` ou `tabela2`). Imprime o valor, a tabela onde está armazenado, e o índice do valor. O formato de saída é `valor, Tn, índice`, onde `n` é o número da tabela.

```
1 void print_tab(struct Hash *table);
```

2.4 Busca

A função `buscar` tem como objetivo localizar a posição de uma chave `k` dentro de uma estrutura de hash duplo.

- A função calcula o índice na `tabela1` utilizando a função de hash `h1(k)`. Se o espaço está vazio (`-1` ou `-2`), portanto, a chave não está presente na tabela. Nesse caso, a função retorna `-1` imediatamente, já que a chave também não estará na `tabela2`. Caso o valor armazenado corresponda à chave `k`, a função retorna o índice calculado.
- Se a chave não foi encontrada na `tabela1`, a função calcula o índice na `tabela2` utilizando a função de hash `h2(k)`. Se o espaço está vazio (`-1`), então a chave não está presente na `tabela2` também. Nesse caso, a função retorna `-1`. Se o valor armazenado corresponde à chave `k`, a função retorna o índice calculado.

```
1 int buscar(int k, struct Hash *table);
```

3 Conclusão

A implementação do Cuckoo Hashing oferece uma solução eficiente para armazenamento e recuperação de dados, garantindo buscas em tempo $O(1)$. Essa abordagem aproveita duas funções de hash independentes para determinar posições alternativas de armazenamento, assegurando que cada elemento tenha apenas dois possíveis locais a serem verificados.

O algoritmo demonstra a força do equilíbrio entre simplicidade e eficiência, sendo uma excelente escolha para aplicações que exigem operações rápidas de busca, inserção e exclusão com um número limitado de colisões.